Abstract

A language server is a crucial component of a modern development environment. In this work I propose a framework for implementing a language server with minimal effort, minimizing code duplication. The implementation relies heavily on Free-Foil, a library that uses Haskell's advanced type system to represent abstract syntax. Free-Foil separates names and their occurrences from the language-specific signature, making most algorithms agnostic—independent of the target language. The resulting language server stores the development environment's configuration, updates it on files changes, shows names' definitions, renames symbols, highlights syntax, performs type-checking, and displays diagnostic errors, warnings, and hover messages. It is possible to launch it in any development environment communicating via the Language Server Protocol.

Chapter 1

Introduction

Language servers are an essential component of modern development environments, providing rich code intelligence features such as code completion, navigation, error checking, and refactoring support. The Language Server Protocol (LSP) [1], introduced by Microsoft, has standardized communication between language servers and editors, allowing a common interface for diverse programming environments. The implementation of a language server from scratch is connected to a pile of boilerplate code, similar and repetitive algorithms for every feature, which makes the job slow, and more code usually means more bugs that are harder to identify later. I wish to make the production of a minimal language server a simpler job by moving as much processing as possible to an agnostic framework which relies on safe and predictable machinery. Hereby, I present a Haskell framework that, given a Free-Foil abstract syntax and a set of functions depending on the language-specific signature, produces a language server communicating via LSP able to store the current environment's configuration, update it on files changes, rename symbols, navigate to a symbol's definition, perform type-checking, and present info, diagnostic, and warnings messages.

1.1 Binding and Scopes

One of the most persistent aspects of language server implementation, as well as in compiler implementation, is to correctly manage variable bindings and scopes. In the following Kotlin example we cannot simply replace x with y in the a's body when calling a(y) because that would mix the local variable x with the free y.

```
val y = 1
fun a (x: Int): (Int) -> Int = { y -> x + y }
a(y)
```

I used Free-Foil [2] to handle bound identifiers. It uses free scoped monads [3], but relies on the approach presented in the Foil [4]. Besides being safe and efficient, Free-Foil allows to generalize the Abstract Syntax Tree (AST), which means the implementation of navigation to a symbol's definition, renaming, or marking errors regarding scoped variables could be agnostic, thus, provided for any language with minimal effort.

1.2 Configuration

1.3 Contribution

Chapter 2

Literature Review

2.1 Name Capture

The problem of name capture is fundamental in programming language implementation, theorem proving, and formal verification systems. When substituting expressions containing bound variables, naïve replacement can lead to variable capture, where a free variable in the substituted term becomes bound, changing the meaning of the expression. For example, in lambda calculus, substituting y for x in $(\lambda \mathrm{y.x})$ would incorrectly yield $(\lambda \mathrm{y.y})$ if performed naïvely. Various approaches to preventing this issue have been developed, each with trade-offs between safety, efficiency, and expressiveness.

The simplest approach, often called "naïve substitution," [5] uses explicit variable names and requires careful renaming during substitution to avoid capture. While intuitive, this approach is error-prone and inefficient as it requires maintaining and checking against a global context of variable names. De Bruijn indices [6] offer an alternative by replacing names with numeric indices representing binding distance. This solves name capture but creates other issues:

terms become difficult to read, and substitution still requires traversing the entire abstract syntax tree (AST) to adjust indices.

Higher-Order Abstract Syntax (HOAS) [7] leverages the host language's binding mechanisms, providing excellent safety but often at the cost of expressiveness when implementing algorithms that need to inspect or transform bound variables. The "rapier" technique used in the Glasgow Haskell Compiler [8] offers better efficiency by maintaining a stateless local scope and performing capture-avoiding substitution in one pass. However, as noted by Maclaurin et al. [4], it lacks type safety, making it easy to forget scope extensions or necessary renamings when implementing complex language features.

2.1.1 Intrinsic Scoping

Intrinsic scoping addresses the safety issues of previous approaches by encoding scope information directly in types. With this technique, the type system ensures that all terms are well-scoped by construction, making ill-scoped terms impossible to express. Bird and Paterson's approach [9] uses nested data types to represent De Bruijn indices at the type level. For Haskell:

```
data Exp a
  = Var a
  | App (Exp a) (Exp a)
  | Lam (Exp (Maybe a))
```

Here, the type parameter changes when going under a binder (using Maybe), enforcing proper scope extension. While this guarantees safety, it still requires traversing and updating all variables during substitution, impacting efficiency.

Additionally, the deep polymorphic recursion consumes significant memory and complicates implementing algorithms that work under binders.

Kmett's bound library [1] extends this approach with more flexible representations like locally nameless [10] and co-de-Bruijn indices [11]. These variations improve ergonomics but still face fundamental efficiency limitations due to their reliance on explicit index manipulation.

2.1.2 The Foil

The Foil [4] resolves the efficiency-safety trade-off by combining the stateless, single-pass algorithm of the rapier with strong type-level guarantees. It implements the Barendregt convention [12], claiming all bound variables to be distinct and different from free variables, with type safety. The Foil achieves this by indexing all expressions, names, and scopes with phantom a type parameter S that represents the current set of in-scope variables. S is defined as data S = VoidS

So every entity in the Foil's representation is scope-indexed by a kind S:

- newtype Name (n :: S) is the type of a name that belongs to the scope n. The name's underlying representation is a machine-sized integer.
- newtype Scope (n :: S) is the type of a scope that contains exactly names of type Name n. Similarly to the rapier, it stores an integer map [13] to lookup and extend the scope.
  - if scope1 :: Scope n and scope2 :: Scope n, then scope1 == scope2
  - if scope :: Scope n and x : Name n, then x `member` scope == True.

[1] https://hackage.haskell.org/package/bound

- newtype NameBinder (n :: S) (l :: S) is the type of a binder that introduces a single new name. It extends the scope indexed by n into the scope indexed by l with the name contained in it. New instances of NameBinder n l can only be created with a withFresh function, which introduces new instances of S-indexed types with rank-2 polymorphism:

```
withFresh :: Distinct n
  => Scope n
  -> (forall l. (Distinct l, Ext n l) => NameBinder n l -> r)
  -> r
```

With these data types, a simply-typed lambda calculus can be represented as the following:

```
data Expr n where
  Var :: Name n -> Expr n
  App :: Expr n -> Expr n -> Expr n
  Lam :: NameBinder n l -> Expr l -> Expr n
```

Here, every expression is indexed with a scope s, as well as variables, which are represented as Name n. It is noteworthy that the body of the lambda expression is indexed with a different scope l, which is exactly the scope NameBinder n l creates. Thus, when going under a binder, the type of the sub-expression changes, imposing a type-level constraint on all the terms within it.

When implementing substitution, the type system ensures:

- Substitution is applied to all sub-terms
- Scope is properly extended when entering binders

- Variables are correctly renamed to avoid capture

The Foil's efficiency comes from its Sinkable type class, which allows "sinking" expressions into extended scopes:

```
sink :: (Sinkable e, Distinct n, Ext n l) => e n -> e l
sink = unsafeCoerce
```

Since the Foil uses name-based representation rather than indices, and the underlying representation of names remains unchanged between scopes, this operation is a type-safe zero-cost coercion. This contrasts with De Bruijn-based approaches that must traverse and increment every index.

Despite its advantages, the Foil requires manually writing syntax representations and substitution implementations for each language, creating boilerplate code and limiting reusability across different languages.

2.1.3 Free Foil

Free Foil bridges the gap between generic syntax representation and efficient, type-safe substitution. It combines Kudasov's free scoped monads [3] with the Foil to automatically generate well-scoped syntax with capture-avoiding substitution from a language's second-order signature. The core idea is to parameterize abstract syntax additionally by a bifunctorial signature sig that distinguishes between regular sub-terms and scoped sub-terms (those that may access newly bound variables) and, additionally, by the binder:

```
data ScopedAST binder sig n where
  ScopedAST :: binder n l -> AST binder sig l -> ScopedAST binder sig n
```

```
data AST binder sig n where
  Var :: Foil.Name n -> AST binder sig n
  Node
    :: sig (ScopedAST binder sig n) (AST binder sig n)
    -> AST binder sig n
```

Given a bifunctorial signature sig, the AST data type automatically generates the complete syntax with proper scoping. For simply-typed lambda calculus:

```
data ExprF scope term
  = AppF term term
  | LamF scope
type Expr = AST NameBinder ExprF
```

CoSinkable typeclass introduces coSinkabilityProof to support patterns. Given a renaming function and a pattern, it provides a renaming function for the variables bound by the pattern, and a refreshed version of the pattern, hence, making sure variables in patterns are refreshed properly to avoid name capture.

The key innovation is that Free Foil inherits the Foil's efficient substitution algorithm while maintaining complete type safety. The Sinkable instance for AST enables zero-cost scope extensions, and capture-avoiding substitution is derived generically:

```
newtype Substitution (e :: S -> Type) (i :: S) (o :: S)
  = UnsafeSubstitution (IntMap (e o))
substitute :: (Bifunctor sig, Distinct o, CoSinkable binder, SinkableK binder)
  => Scope o
```

```
-> Substitution (AST binder sig) i o
-> AST binder sig i
-> AST binder sig o
```

Unlike free scoped monads that use nested De Bruijn indices, Free Foil eliminates the need for explicit index manipulation, dramatically improving performance while maintaining the same level of type safety. Benchmarks show that while Free Foil is slightly slower than direct Foil implementations due to the additional layer of abstraction [2], it outperforms nested De Bruijn approaches by orders of magnitude for complex terms.

Additionally, Free Foil supports a template-based approach where scope-safe representations can be automatically derived from naïve AST definitions using Template Haskell, allowing seamless integration with parser generators like BNF Converter (BNFC) [14].

2.2 Language Server Protocol (LSP)

The Language Server Protocol [1] standardizes communication between language servers and editors, allowing a common interface for diverse programming environments.

Rust Language Server

Rust-analyzer [2] represents a modern approach with internal architecture structured as a set of libraries for analyzing Rust code, emphasizing performance

[2] https://github.com/rust-lang/rust-analyzer/tree/master

and correctness. Its design focuses on providing comprehensive language understanding through incremental analysis.

Go Language Server

Gopls [3] aims to be a full LSP implementation that standardizes features according to the LSP specification while maintaining cleanliness and robustness. It has achieved community-wide adoption as the definitive Go language server, supported by the entire Go ecosystem.

Python Language Server

Python implementations often build upon existing analysis libraries like Jedi [4], demonstrating how LSP servers can leverage mature language analysis tools while providing standardized editor integration.

Haskell Language Server

The Haskell Language Server (HLS) [5] serves as the primary GHC-oriented LSP implementation for Haskell. It functions as an integration layer that combines multiple analysis engines and plugins to provide comprehensive language support. HLS architecture employs a plugin system that allows modular extension of functionality

[3] https://go.dev/gopls/
[4] https://pypi.org/project/jedi/
[5] https://github.com/haskell/haskell-language-server

2.2.1 Libraries for LSP

Some frameworks propose a general language server implementation following Microsoft LSP, simplifying language server development for any target.

Eclipse LSP4J

LSP4J [6] is a generic LSP implementation framework allowing to implement language servers in Java. Metals for Scala [7] and Kotlin Language Server [8] both base on LSP4J.

Haskell LSP

The haskell/lsp [9] library family (including lsp-types and lsp-test) provides Haskell with low-level protocol handling capabilities. This ecosystem offers type-safe implementations of LSP message types and communication handling. The library has been battle-tested through its use in production systems like Haskell Language Server (HLS).

TODO: describe the LSP functionality relevant to the implementation

2.2.2 Comparative Analysis

Advantages of Current Approaches

- Standardization: LSP enables editor-agnostic tooling development

[6] https://github.com/eclipse-lsp4j/lsp4j
[7] https://github.com/scalameta/metals
[8] https://github.com/Kotlin/kotlin-lsp
[9] https://github.com/haskell/lsp

- Modular Architecture: Modern implementations like HLS support plugin-based extensibility
- Performance Focus: Leading implementations prioritize incremental analysis and caching strategies
- Community Consolidation: Languages like Go have achieved unified implementations (gopls)

Disadvantages and Challenges

- Performance Issues: Haskell implementations face challenges with Template Haskell and large projects
- Concurrency Limitations: Some LSP servers struggle with concurrent usage patterns
- Protocol Complexity: Full LSP specification compliance requires significant implementation effort
- Resource Consumption: Language servers can consume substantial CPU and memory resources during analysis

2.3 Conclusion

This literature review has explored two complementary areas: techniques for safe and efficient name handling in language implementation, and the practical ecosystem of LSP tooling.

On the theoretical side, approaches to name capture have evolved from error-prone manual renaming to intrinsically scoped systems like the Foil and Free

Foil. These modern solutions enforce scoping correctness at the type level while enabling zero-cost substitutions—offering both safety and performance.

On the practical side, LSP implementations such as gopls, rust-analyzer, and the Haskell Language Server demonstrate how standardized protocols enable powerful, editor-agnostic developer tooling. Many of these servers—like Python's—rely on libraries such as Jedi for static analysis and autocompletion.

Yet a gap remains: despite advances in type-safe scoping, most LSPs still use traditional, extrinsically scoped ASTs, leaving them vulnerable to subtle binding bugs and inefficiencies. Integrating intrinsically scoped representations like the Foil into language servers presents a promising path toward more robust and performant tooling—especially for languages with rich binding semantics.

This thesis builds on that opportunity, aiming to bridge formal scoping guarantees with real-world LSP development.

Chapter 3

Methodology

3.1 Design Requirements

The goal of my research is to develop an out-of-box solution for a minimal language server. Given a Foil AST and some signature-dependant functions, it has to compile into a binary communicating via LSP. I would also like to provide a small example-project demonstrating the integration of such a language server into the Visual Studio Code (VS Code) [1], an open-source code editor.

The main criterion is the simplicity for an end user. This also means requiring as less boilerplate code as possible. So that a person wishing to create a proof assistant or a simple programming language for educational or research purposes could have provided the minimal set of functions, while all the heavy-lifting would have lied on the agnostic languages server's shoulders. Specifically, the implementation of LSP methods for symbols definition, renaming, type-checking, and code highlighting. All of the methods above could be implemented by juggling a finite amount of functions traversing the AST.

[1] https://code.visualstudio.com/

3.1.1 Deprioritized Criteria

The project does not focus on substituting complex enterprise-level language servers, nor does it provide a "swiss knife" for any level of complexity projects. Hence, the performance and extensibility are deprioritized. Though, this doesn't mean the behavior of a resulting language server should be non-customizable and slow. Rather it only implies the performance and extensibility must not come in contradiction with the prioritized criteria.

3.1.2 Evaluation Criteria

To assess whether the proposed language server framework meets its design goals, the following criteria will be used:

1. **Integration Simplicity**: The number of functions an end user must implement to create a working language server. A successful implementation should require no more than six core functions (as defined in $\mathrm{ALSConfiguration}$) with minimal boilerplate code.

2. **Feature Completeness**: The framework must support essential LSP features including *go-to-definition*, *find-references*, *rename-symbol*, *hover*, and *syntax highlighting*. Each feature should function correctly when the corresponding user-provided functions are implemented.

3. **Correctness**: Symbol resolution must accurately trace variable definitions to their usages within the correct scope. The $\mathrm{TermTelescope}$ structure should correctly represent scope extensions and variable bindings without false positives or negatives.

4. **Language Agnosticism**: The framework should work with at least two distinct language implementations without modification to the core server logic. This demonstrates the binder and sig parameterization achieves true abstraction.

5. **Response Time**: While performance is deprioritized, LSP requests should complete within 500ms for typical source files (1000 lines) to maintain acceptable user experience in VS Code.

6. **VS Code Integration**: The example project must demonstrate successful integration with Visual Studio Code, with all supported LSP features accessible through the editor's standard interface.

These criteria will be evaluated through the implementation of the example project mentioned in Section 3, as well as through unit tests for core functionality such as scope tracking and name resolution.

3.2 Language Server Methods

3.2.1 Building a Tree

The first operation required for any language server is to build the AST, since it is essentially what is going to be analyzed throughout the course of a user's session. Given a language's file extension, finding certain files, converting them into ASTs, and even tracking modifications is relatively easy. However, parsing the AST must be a user's task since the language server is supposed to know nothing about the syntax of the given language.

Some languages may represent programs as a list of declarations. From the scope-tracking view, these are programs where the scopes gets injected manually to an expression according to the whole project's context. The program, hence, is defined as a set of expressions—not a nested structure. Scope management in these scenarios becomes more sophisticated compared to the nested one-expression case. Yet it's still possible to produce a Free Foil AST leaving a single expression representing a whole program afterwards, turning it into a list of expressions each with a distinct scope is also a solution. In terms of an agnostic language server producing a list of trees from a given String input is a cheap yet powerful modification comparing to limiting it to a single element allowing for more user customizations without a requirement to extend the second-order abstract syntax signature.

The function producing Free Foil ASTs from a given input is defined as the following:

```
type FunBuildASTs binder sig
  = String
  -> [SomeScopedAST binder sig]
```

SomeScopedAST is parametrized by a binder and sig but it also introduces an existential variable n which makes any function referring to the SomeScopedAST independent of the concrete phantom type parameter n while also preserving the relation of the AST binder sig n to the scope Scope n.

```
data SomeScopedAST binder sig where
  SomeScopedAST :: Distinct n
    => Scope n
```

```
  -> AST binder sig n
  -> SomeScopedAST binder sig
```

3.2.2 Finding a Node

The first operation required for any language server is to map the location of the user cursor, the line and column numbers, to a node in the AST. The BNFC represents the AST nodes as Haskell Functors parameterized by their location like so:

```
type BNFC'Position = Maybe (Int, Int)
type ASTNode = ASTNode' BNFC'Position
data ASTNode' a = AnASTNode a [ASTChild' a]
  deriving
    ( Eq
    , Ord
    , Show
    , Read
    , Functor
    , Foldable
    , Traversable
    )
```

These locations can be stored in the Free Foil AST's signature annotations. However, the BNFC does not provide the ending positions for nodes, even for identifiers. So for an end user finding a name means traversing the AST and picking the narrowest child for which the positions range must be somehow computed

first. The good news is the naïve implementation for a symbol written horizontally having the form of ((x, y), (x + symbolLength, y)) unlocks functionality enough for a bunch of language server features related to names!

Another struggle imposed by the Free Foil design is the disability to provide an annotation for a Var. Since only Nodes are annotated by the signature, one can only have access to the Name represented by a given variable, which is essentially just an integer value. Even though it makes impossible to use Foil Vars for anything but scope tracking, the easiest solution would be to come up with a variable-Node wrapper. It is a user's task to produce concrete syntax and convert it to a Foil AST, hence, the user could've wrapped Vars into specific variable-Nodes to make them locatable. It is not so much an unsafe decision as it is unreliable for an end user to always believe a random Node might always be a Var in disguise. Luckily, it is up to a user to decide how to exactly make the Nodes locatable and to pick the proper representation.

The function searching for nodes is defined as:

```
type FunFindNarrowest binder sig
  = (Int, Int)
  -> SomeScopedAST binder sig
  -> Maybe (SomeScopedAST binder sig)
```

At the same time an auxiliary function is used to extract a name from any node if the narrowest node happens to be a Var or a Node dedicated to directly wrap a Free Foil variable, thus, making it locatable as discussed earlier:

```
type FunExtractName binder sig
  = SomeScopedAST binder sig
  -> Maybe (SomeName binder sig)
```

3.2.3 Tracing a Name

To find any symbol's definition and its further usages requires traversing the AST and collecting Nodes with the consequent binders and, eventually, Vars referring to the queried name. The structure storing these relations can have a form:

```
data TermTelescope binder sig n where
  LeafTerm :: Distinct n
    => AST binder sig n
    -> TermTelescope binder sig n
  NodeTerm :: (Distinct n, DExt n l, CoSinkable binder)
    => AST binder sig n -- The term
    -> binder n l -- The term's binder
    -> [TermTelescope binder sig l] -- Scoped [NodeTerm]s
    -> TermTelescope binder sig n
```

One TermTelescope matches a single expression containing all nested Nodes, extending the scope, and variables, supposedly, as leaves. The end user builds a TermTelescope for each expression, and later the structure is traversed to match a specific Name. TermTelescopes allow to select only nodes extending the scope and variables, which can be used to find a name's definition or trace its usages generally. Thus, one TermTelescope can be used to build a structure focusing on the definition of some name. Some data types wrap the scope phantom type parameter to indicate the scope of the resulting expression is of any scope.

```
data SomeName binder sig where
  SomeName :: Distinct n
```

```
  => Name n
  -> SomeName binder sig

data Family binder sig = Family
  { parent :: SomeScopedAST binder sig
  , children :: [SomeScopedAST binder sig]
  , name :: SomeName binder sig
  }
```

The binder concrete implementation is unknown, in contrast, it is only said to be CoSinkable making it available to extend any scope independent of the language's specification.

In a program one expression can produce multiple branches of scope-extension resulting in a list of TermTelescopes. For instance, if the expression is a pair of lambdas—(λ a . a, λ b. b)—the AST gets divided in two TermTelescopes right away. Thus, a function building TermTelescopes from a single AST may look like:

```
type FunBuildTelescopes binder sig
  = forall n . Distinct n
  => Scope n
  -> AST binder sig n
  -> [TermTelescope binder sig n]
```

The function introduces an existential phantom type parameter that signifies the AST binder sig n exists in the Scope n. Moreover, in the resulting list of TermTelescopes, each shares the same Scope n. It is impossible for a TermTelescope in this scenario to be parametrized by a different scope type parameter as

this would have meant another NodeTerm parametrized by a narrower scope with a consequent binder introduced the name. Moreover, the function operates with a specific Scope and AST in contrast to passing a SomeScopedAST it resembles so much to explicitly parametrize them with the same type parameter as the resulting TermTelescopes to foster the correct and predictable scope-management.

3.2.4 Node Into a Range

TODO

3.2.5 Type Checking

TODO

3.2.6 Code Highlighting

TODO

3.3 Language Server Configuration

The configuration required to run a language server is defined as:

```
data ALSConfiguration binder sig = ALSConfiguration
  { fileExtension :: String
  , buildAsts :: FunBuildASTs binder sig
  , findNarrowest :: FunFindNarrowest binder sig
  , extractName :: FunExtractName binder sig
  , buildTelescopes :: FunBuildTelescopes binder sig
```

```
, findRange :: FunRange binder sig
}
```

Chapter 4

Implementation

4.1 LSP Interface

The Haskell LSP library [1] allows to easily run a language server and define its behavior using a list of handlers. A handler is a function acting in response to a request body. The library defines a finite amount of events which can be triggered by a notification or manual request, which, in terms of handlers are all, eventually, just events triggering a consequent response function. The project uses the following handler-functions:

- Initialized is triggered on the language server initialization
- WorkspaceDidChangeWatchedFiles is triggered when files that are currently in focus in the editor change
- TextDocumentDefinition is triggered when the user requests to navigate to a definition of a given symbol

[1] https://github.com/haskell/lsp

- TextDocumentRename is triggered when the user points at a symbol and wishes to rename it

4.1.1 Building a Tree

The LSP.Server.Core module provides a function fetching the root path of a project. Then by using globDir defined in System.FilePath.Glob.Directory module it is possible to filter out files with the extension related to the given language and by using FunBuildASTs produce ASTs for each file.

4.2 Caching

The ASTs produced on initialization are cached for later usage. Every watched files change triggers ASTs rebuild.

The cache is defined as a separate module independent of Free Foil which is deliberately done to foster modular design and future extensibility to adapt caching to more complex scenarios. Caching is implemented using Software Transactional Memory (STM) [2]. The program's environment stores a mutable variable LangStore ast parametrized by the final AST type—SomeScopedAST binder sig in the current implementation. LangStore stores a Map connecting FilePaths to LangProgramStores, each, essentially, storing the list of ASTs:

```
newtype LangProgramStore ast = LangProgramStore
  { langAst :: [ast]
  }
```

[2] https://hackage.haskell.org/package/stm

```
newtype LangStore ast
  = LangStore (Map FilePath (LangProgramStore ast))

newtype LangEnv ast = LangEnv
  { store :: TVar (LangStore ast)
  }
```

The caching module implements two functions reading and writing to the environment as well as the LSP parametrizing the LspT.

```
type LSP ast = LspT () (ReaderT (LangEnv ast) IO)
```

4.3 Connecting Families

Whether it is finding a symbol's definition or tracing a symbols usage, a Family must be built first, as described in the 3.2.3. A function dedicated to building a family with respect to a given Name follows:

```
nameFamily :: Distinct n
  => FunExtractName binder sig
  -> SomeName binder sig
  -> Scope n
  -> TermTelescope binder sig n
  -> Maybe (Family binder sig)
```

Since NodeTerms contain a CoSinkable binder, the scope can be dragged all the way through the TermTelescope's structure until a binder injecting the queried name is found. If the binder is not adding the name, its child carries on the

search, and later—siblings. Technically, it is simply a Depth First Search (DFS) algorithm. Once the parent—a node establishing the given name—is found, scope-management is still required to produce the SomeScopedAST from the LeafNode. If the Free Foil AST was built correctly, succeeding TermTelescopes will introduce names distinct from the queried, thus, additional checks on the name-in-scope membership after finding the parent are not required.

4.3.1 Names Collisions

The possibility of multiple alternative scope extensions as discussed in the 3.2.3 means multiple TermTelescopes can be produced for a single expression. However, it also means the Foil NameBinder operating with an integers set under the hood is going to extend the scope identically for every AST branch. As depicted in the example, a Haskell tuple of three lambdas extends the Scope n three times introducing three Names all parametrized by a different type parameter, while under the hood all three equal to the same integer value, which also means all three integer sets representing Scope l1, Scope l2, and Scope l3 are equal.

```
myFunction = -- Scope n
  ( \x -> doSmth1 x -- Scope l1
  , \y -> doSmth2 y -- Scope l2
  , \x -> doSmth3 x -- Scope l3
  )
```

In practice, for the above example the nameFamily is going to falsely pick the first NodeTerm introducing the given Name resulting in 33% success rate. Comparing raw String names injected in each node would still confuse the first and third branches, so it's not a solution either.

The simplest solution is to check whether a given subtree contains the required Var. Since the language server requires all symbols to be locatable to use the FunFindNarrowest, the queried name must be present in the given subtree. From the user's perspective, the AST may be traversed several time for each option leading to the $O(n^2)$ algorithm complexity in the worst case scenario. Yet if the user's implementation can cut off searches that for sure do not lead to a given position, the algorithm completes in $O(n)$ even in the worst case.

An alternative solution is radical as it implies to adapt the internal Foil machinery to the following case. The Scope can possibly exclude some Names from itself marking them as "taken". It must not even be an additional integer set but rather an offset representing how much names were scoped in the preceding cases of Free Foil AST creation. In a signature with pairs, the modification would mean building the Pair node required converting cases one by one, which makes each case in the node implicitly dependant on the scopes of other alternative cases. That makes it impossible to parallelize the conversion of a concrete syntax tree to the abstract representation.

```
toAst :: Distinct n
  => OffsetScope n
  -> Map String (Name n)
  -> RawTerm
  -> (MyAst n, Int)
toAst scope env = \case
  ... -- Other AST to Foil AST converting cases
  RawPair l r ->
    let (astL, lShift) = toAst scope env l
```

```
    (astR, rShift) = toAst (shiftScope scope lShift) env r
  in (Node (Pair astL astR), rShift)
```

This sophisticated scope management can be applied only when building the AST. The ShiftScope assumed to be the default Foil Scope with an additional integer used in the customized version of rawFreshName.

```
rawFreshName scope offset = if IntSet.null scope
  then offset
  else offset + 1 + (IntSet.findMax scope)
```

Hence, if the scope was managed correctly using shifts, the final tree will have no name collisions.

4.4 Example Project

I tested the implementation of the agnostic language server by applying it to two toy programming languages—LambdaPi and FLAN (meaning Functional LANguage). Flan was required to test type checking and more complex name tracing algorithms.

4.4.1 Grammar and Signature

Flan is a strictly typed functional programming language with the following signature:

```
data FlanType a
  = IntType a
```

```
  | StrType a
  | BoolType a
  | FunType a (FlanType a) (FlanType a)
  | PairType a (FlanType a) (FlanType a)
  | UnknownType a String

data FlanPattern a
  = PatternWildcard a
  | PatternVar a String
  | PatternPair a (FlanPattern a) (FlanPattern a)

data FlanF a s t
  = VarF a t String
  | AppF a t t
  | LamF a (FlanPattern a) (PatternType a) s
  | LetF a (FlanPattern a) (PatternType a) t s
  | PairF a t t
  | IfF a t t t
  | ConstF a FlanConst
  | ErrorF a String
```

Every expression is parametrized by an annotation. Initially, it only stores raw BNFC location of a given node. Later, annotations can be used to match types during type checking.

Instead of the default NameBinder, Flan's AST uses NameBinders as the binder. Free Foil provides a mechanism to bind zero or many names for one node,

thus, allowing to bind patterns of any complexity easily.

4.4.2 Visual Studio Code Demo

I developed a simple VSCode demonstration project to test the compiled language server. With some ALSConfiguration, the program can be built as an executable and hosted in a VSC extension.

...

Bibliography cited

[1] *Language Server Protocol*. [Online]. Available: https://microsoft.github.io/language-server-protocol/.

[2] N. Kudasov, R. Shakirova, E. Shalagin, and K. Tyulebaeva, *Free foil: Generating efficient and scope-safe abstract syntax*, 2024. arXiv: 2405.16384 [cs.PL]. [Online]. Available: https://arxiv.org/abs/2405.16384.

[3] N. Kudasov, *Free Monads, Intrinsic Scoping, and Higher-Order Preunification*, To appear in *TFP 2024*, Orange, NJ, USA, 2024. arXiv: 2204.05653 [cs.LO].

[4] D. Maclaurin, A. Radul, and A. Paszke, "The foil: Capture-avoiding substitution with no sharp edges," in *Proceedings of the 34th Symposium on Implementation and Application of Functional Languages*, ser. IFL '22, Copenhagen, Denmark: Association for Computing Machinery, 2023, ISBN: 9781450398312. DOI: 10.1145/3587216.3587224. [Online]. Available: https://doi.org/10.1145/3587216.3587224.

[5] E. Kmett, *Bound*, Accessed: 2023-01-21, Dec. 2015. [Online]. Available: %7Bhttps://www.schoolofhaskell.com/user/edwardk/bound%7D.

[6] N. de Bruijn, "Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser the-

orem," *Indagationes Mathematicae (Proceedings)*, vol. 75, no. 5, pp. 381–392, 1972, ISSN: 1385-7258. DOI: https://doi.org/10.1016/1385-7258(72)90034-0. [Online]. Available: https://www.sciencedirect.com/science/article/pii/1385725872900340.

[7] F. Pfenning and C. Elliott, "Higher-order abstract syntax," in *Proceedings of the ACM SIGPLAN'88 Conference on Programming Language Design and Implementation (PLDI), Atlanta, Georgia, USA, June 22-24, 1988*, R. L. Wexelblat, Ed., ACM, 1988, pp. 199–208. DOI: 10.1145/53990.54010.

[8] S. Peyton Jones and S. Marlow, "Secrets of the glasgow haskell compiler inliner," *Journal of Functional Programming*, vol. 12, pp. 393–434, Jul. 2002. [Online]. Available: https://www.microsoft.com/en-us/research/publication/secrets-of-the-glasgow-haskell-compiler-inliner/.

[9] R. S. Bird and R. Paterson, "De bruijn notation as a nested datatype," *J. Funct. Program.*, vol. 9, no. 1, pp. 77–91, Jan. 1999, ISSN: 0956-7968. DOI: 10.1017/S0956796899003366. [Online]. Available: https://doi.org/10.1017/S0956796899003366.

[10] A. Charguéraud, "The locally nameless representation," *J. Autom. Reason.*, vol. 49, no. 3, pp. 363–408, 2012. DOI: 10.1007/S10817-011-9225-2. [Online]. Available: https://doi.org/10.1007/s10817-011-9225-2.

[11] C. McBride, "Everybody's got to be somewhere," in *Proceedings of the 7th Workshop on Mathematically Structured Functional Programming, MSFP@FSCD 2018, Oxford, UK, 8th July 2018*, R. Atkey and S. Lindley, Eds., ser. EPTCS, vol. 275, 2018, pp. 53–69. DOI: 10.4204/EPTCS.275.6. [Online]. Available: https://doi.org/10.4204/EPTCS.275.6.

[12] H. (Barendregt and E. Barendsen, "Introduction to lambda calculus," *Nieuw archief voor wisenkunde*, vol. 4, pp. 337–372, Jan. 1984.

[13] C. Okasaki and A. Gill, "Fast mergeable integer maps," in *Workshop on ML*, 1998, pp. 77–86.

[14] M. Forsberg and A. Ranta, "Demonstration abstract : Bnf converter," 2004. [Online]. Available: https://api.semanticscholar.org/CorpusID:9335858.